FINAL PROJECT PRESENTATION · ADVANCED ALGORITHMS

Unbuffered Permutation Routing on Hypercubes

Algorithms, Theory, and Experimental Evaluation of A*, Entropy-Cycle, and Stochastic Search

Team Members:

C++ Three.js Gemini 3.1 Pro Antigravity

ubperm-routing

CONTENTS

What We'll Cover

PROBLEM DEFINITION

Hypercube Permutation Routing

An **n-dimensional hypercube** has $N = 2^n$ nodes; two nodes share an edge iff their addresses differ in exactly one bit.

Each node holds one packet addressed to a **unique destination** — the initial state is an arbitrary permutation of N packets.

At each step, adjacent nodes may **swap their packets** along a shared edge (deflection routing — no buffers, no queuing).

Goal: reach the identity permutation (packet i is at node i) in the fewest total swaps.

TOPOLOGY

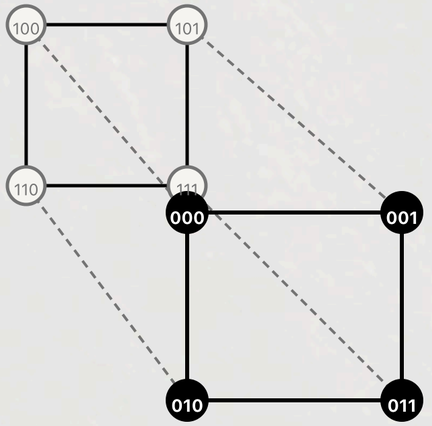
The n-Cube Network

n=3 (3-cube): 8 nodes, 12 edges. Used for exhaustive experiment (8! = 40,320 permutations).

n=4 (4-cube): 16 nodes, 32 edges. Partial sampling used.

Swap along edge (u, v) moves **both packets simultaneously** — one step regardless of direction.

State space grows as **N!** — intractable for BFS beyond N=8.



PART 01

The Algorithms

Three distinct search strategies — from informed heuristics to algebraic theory to probabilistic sampling.

ALGORITHM 01 · A* SEARCH

Informed Search with Hamming Heuristic

$$f(n) = g(n) + h(n)$$

g(n): actual swap cost from initial state to current state n.

h(n) = Hamming heuristic: for each packet, count bits still wrong via `popcount(current XOR target)`; divide sum by 2 (each swap corrects 2 packets).

Heuristic is **admissible** — never overestimates, guaranteeing optimal path length.

Expands exponentially fewer states than BFS; priority queue over f(n) prunes unpromising branches.

BFS (BASELINE)

$$\mathcal{O}(b^D) \approx \mathcal{O}(N!)$$

- Explores uniformly in all directions
- Guarantees absolute optimal path
- Infeasible beyond N=8 — state space explodes
- Memory: $\mathcal{O}(N!)$ — stores all visited states

A* SEARCH

$\mathcal{O}(b^D)$ **worst-case**

- Guided by f(n) = g(n) + h(n) priority queue
- Prunes millions of unpromising branches
- Matches BFS optimal paths in practice (N=8)
- Memory: $\mathcal{O}(b^D)$ — still grows large for $N > 8$

Group Theory: Cycle Decomposition

The routing state is viewed as a **mathematical permutation**; any permutation decomposes uniquely into disjoint cycles.

Swapping packets **within the same cycle** fractures it into two smaller cycles (progress). Swapping **across cycles** merges them (regression).

Minimum transpositions to fully sort from C cycles: $\text{min\_swaps} = N - C$

Algorithm greedily prefers swaps that **increase C** — maximally fracturing large cycles.

Information Theory: Shannon Entropy Tie-Breaking

When multiple swaps tie on cycle-count gain, a **Shannon entropy penalty** breaks ties.

Entropy is computed over the **displacement error distribution** across bit dimensions — high entropy = chaotic scattering.

Combined heuristic: $h = (N - C) + 0.1 \times H$ where H is the dimensional entropy of residual errors.

Synergy: cycle decomposition unknots permutations algebraically; entropy organises the network dimension-by-dimension.

ALGORITHM 03 · STOCHASTIC SEARCH

Algebraic + Probabilistic Sampling

Combines the **Entropy-Cycle heuristic** with **Softmax (Gibbs/Simulated Annealing)** probabilistic selection over candidate swaps.

Memory footprint: $\mathcal{O}(1)$ — no priority queue, no closed-set; walks a single path with probabilistic restarts.

Scales to $N = 64, N = 256$ and beyond — elegantly sidesteps the curse of dimensionality.

Trade-off: slight long-tail deviation in path length compared to exact search; excellent for massive networks where optimal search is infeasible.

ALGORITHM 04 · BEAM SEARCH

Heuristic Path Pruning

Beam Width (k): keeps only the top k most promising states at each depth.

Uses the **Hamming heuristic** to evaluate state promise without expanding the full frontier.

Trades **optimality** for **memory efficiency** — no guarantees of shortest path, but finds a valid routing fast.

Avoids the exponential memory blowup of A* while outperforming pure greedy descent.

ALGORITHMS · SIDE BY SIDE

Complexity & Scalability

ALGORITHM	TIME COMPLEXITY	MEMORY	OPTIMAL PATH?
A* Search	$\mathcal{O}(b^D)$ — heuristic pruned	$\mathcal{O}(b^D)$	✓ Guaranteed
Entropy-Cycle	$\mathcal{O}(b^D)$ — aggressively pruned	$\mathcal{O}(b^D)$	≈ Near-optimal
Beam Search	$\mathcal{O}(k \cdot b)$ — bounded by width	$\mathcal{O}(k \cdot D)$	Fast, not optimal
Stochastic	Near-polynomial empirically	$\mathcal{O}(1)$	~ Close, not exact

PART 02

Experiment Results

Exhaustive evaluation on all 40,320 permutations of the 3-cube and selected cases for 4-cube, and higher dimension.

EXPERIMENTAL SETUP

Evaluation Protocol

Network: 3-cube (n=3), N=8 nodes, 12 edges.

Coverage: All 8! = 40,320 distinct initial permutations evaluated exhaustively via the `test.py` batch script with multithreading.

Metric: number of edge-swap steps (lower = better path quality).

Algorithms compared: A* Search, Entropy-Cycle Search, Stochastic Search, and Beam Search. BFS serves as the optimal baseline; Bitonic Merge Sort as the deterministic non-search baseline.

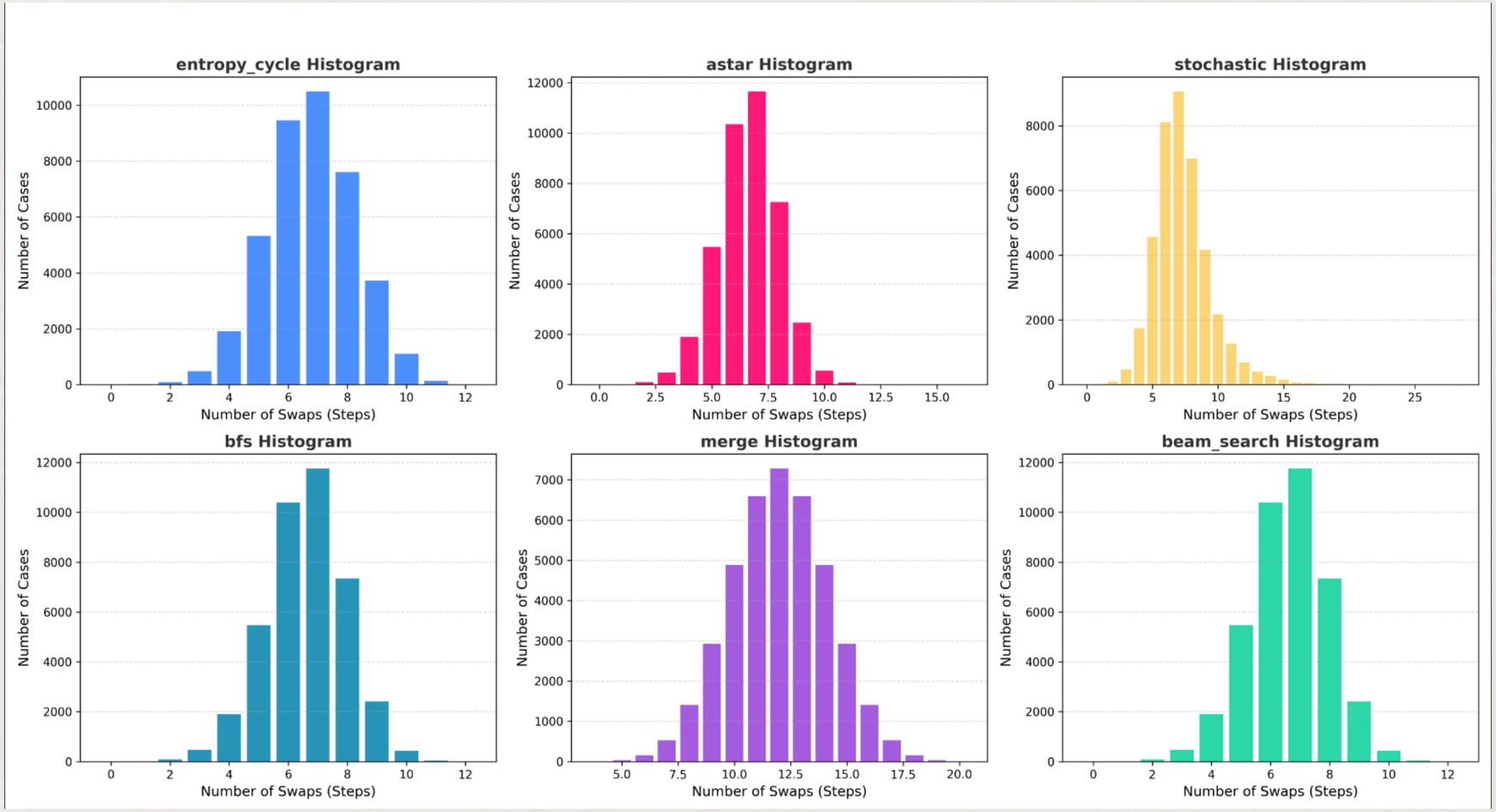
RESULTS · KEY STATISTICS · N=8

Average Steps on 3-Cube

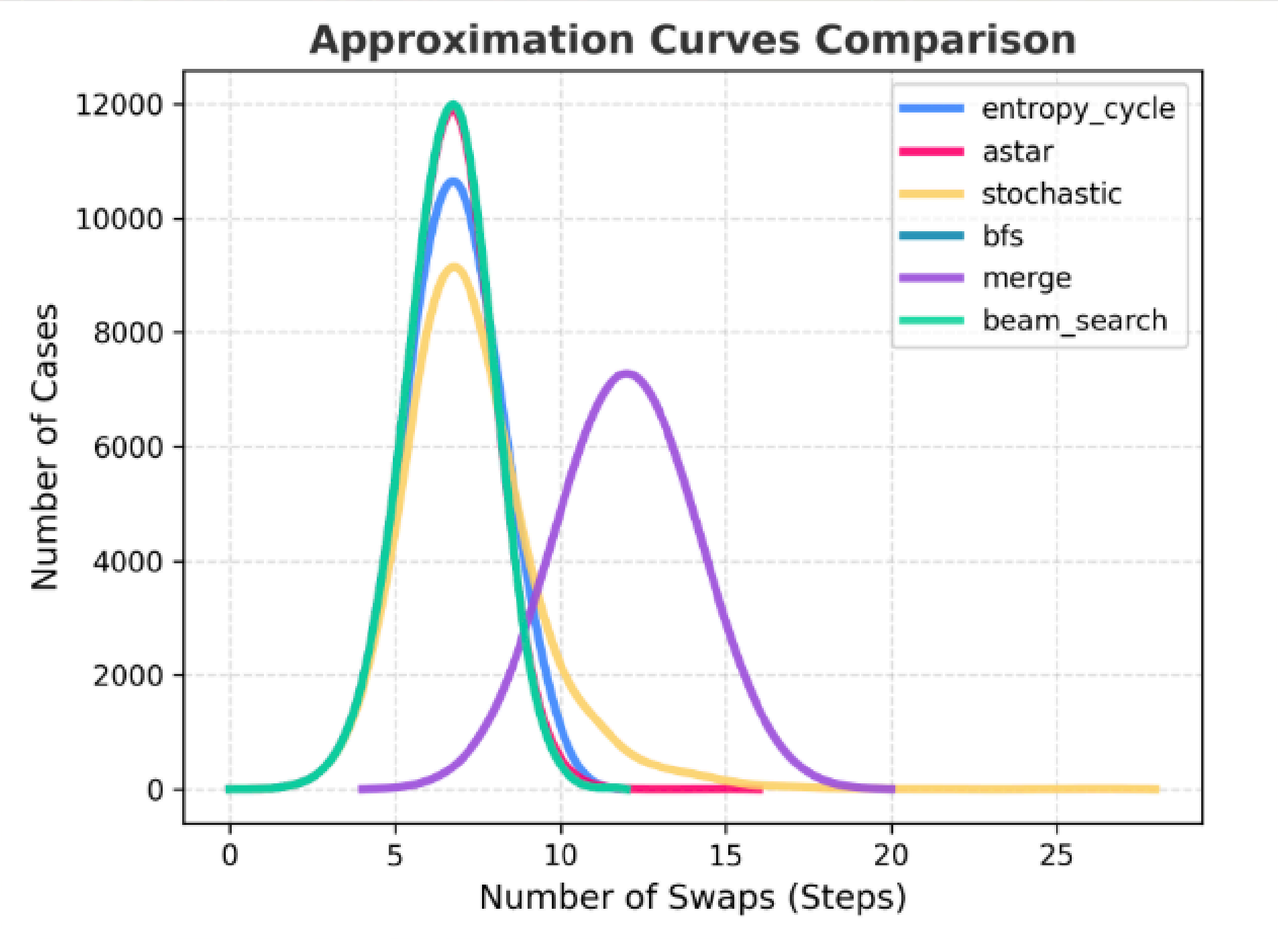
MERGE	STOCHASTIC	ENTROPY-CYCLE	A*	BEAM SEARCH	BFS (OPTIMAL)
12.0	**7.32**	**6.77**	**6.62**	**6.61**	**6.61**
mean steps	mean steps	mean steps	mean steps	mean steps	mean steps

Key insight: A*, Beam Search, and Entropy-Cycle produce path lengths virtually identical to the BFS optimal baseline for N=8.

All Algorithms Swap Steps (N=8)



Step-Count Distributions



RESULTS · KEY STATISTICS · N=16

Algorithm Performance at a Glance

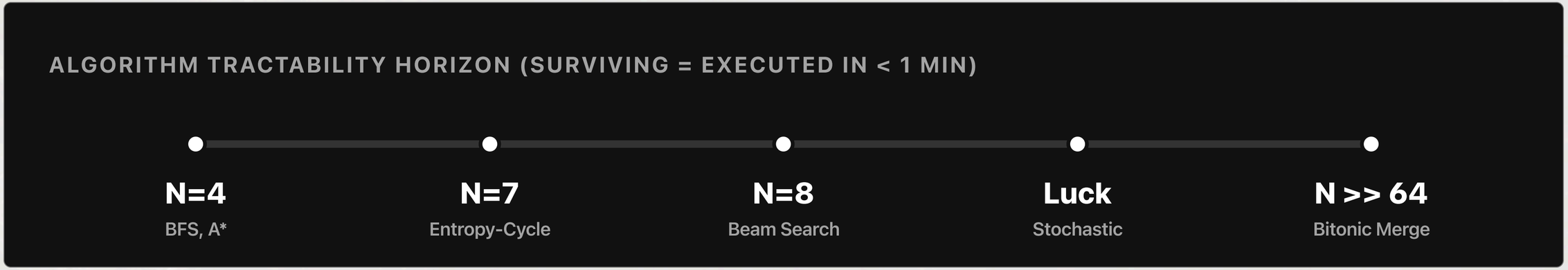
BITONIC MERGE	STOCHASTIC	ENTROPY-CYCLE	A* (OPTIMAL)	BEAM SEARCH
37.5	**23.6**	**15.5**	**15.0**	**14.5**
mean steps	mean steps	mean steps	mean steps	mean steps

Key insight: A* and Entropy-Cycle both achieve near-BFS-optimal step counts. Stochastic trades slight path overhead for $\mathcal{O}(1)$ memory and massive scalability.

The Curse of Dimensionality

For $N \geq 64$ hypercubes, the state space reaches astronomically large numbers — making exact BFS or A* Search entirely intractable.

Memory bounds are the primary bottleneck, as optimal path algorithms like A* queue up exponentially growing frontiers.



SUMMARY & FUTURE WORK

Takeaways

A* Search delivers provably optimal routing on N≤8 with dramatic speedup over BFS.

Entropy-Cycle Search marries group theory and information theory for near-optimal, near-polynomial empirical runtime.

Stochastic Search breaks the memory barrier — $\mathcal{O}(1)$ footprint enables $N = 256$ and beyond.

Future work: adaptive temperature schedules for stochastic; multi-step lookahead for entropy-cycle; GPU-parallelised beam search.

Live Visualization

Scan the QR code below to visit the interactive website for visualizing hypercube permutation routing.

thisiselijah.github.io/ubperm-routing/